



**UNIVERSITI TUNKU ABDUL RAHMAN
FACULTY OF ACCOUNTANCY AND MANAGEMENT
LEE KONG CHIAN FACULTY OF ENGINEERING AND SCIENCE**

Machine Learning Assignment Report

No.	Name	No.id	Trimester
1.	Chin Lok Bin	24UEB00968	Y2S2
2.	Lew Jia Jia	23UEB00361	Y2S2
3.	Tham Min Cong	22UEB04524	Y3S1

1.0 Introduction	3
2.1 Data Description	5
2.1.1 Deepfake Image Detection Dataset	5
2.1.2 DeepDetect-2025	6
2.2 The Reason for Using These Datasets	6
2.3 Exploratory Data Analysis	7
3.0 Methodology	8
3.1 Flowchart of the Training and Development Pipeline	8
3.2 Data Pre-Processing	11
3.3 Feature Selection or Creation	12
3.4 Model Choice	13
3.5 Evaluation Metrics	14
3.6 Methodology Justification	15
4.0 Result and Discussion	17
4.1 Expected Model Performance Summary	17
4.2 Comparative Model Performance Summary	18
4.3 Discussion of Why Models Performed Well or Poorly	19
5.0 Conclusion	20
5.1 Summary of Results and Conclusions	20
5.2 Strengths and Weaknesses of the Methodology	21
5.3 Suggested Possible Improvements	22
Appendix	23

1.0 Introduction

Deepfake image detection is the technique and methods used to identify images that have been artificially generated or manipulated using AI. With deep learning models developments like Generative Adversarial Networks (GANs) and diffusion models, images generated with AI are becoming more realistic, which makes it difficult for humans to distinguish between real and fake visual content. While these technologies are widely used for creative design, entertainment, and accessibility sectors, misuse may also lead to serious concerns such as identity theft, fraud, and reputational damage.

As deepfake image generation is getting easier, the detection of manipulated images has become one of the most important issues. Deepfake images detection systems detect inconsistencies in visual and structural that can evidence artificial manipulation. These include unnatural facial features, inconsistent lighting or shadows, distorted textures, irregular edges, and subtle digital artifacts invisible for the naked eye. Currently, the approaches to detection are grounded on computer vision, forensics, and machine learning models to identify these kinds of anomalies.

The continuous “arms race” between deepfake image generation methods and techniques for detecting them makes the field of their detection change very fast. As generative models continue to produce more realistic images, there is a constant need to develop more robust and generalizable detection methods to counter unknown tampering techniques. Effective deepfake images detection plays an important role in protecting the integrity, maintaining public trust in visual media, and preventing harmful impact of AI generated misinformation.

Given the serious risks that deepfake images pose to individuals, organizations, and society, accurate image-based deepfake detection is extremely important for ensuring digital security and ethical accountability. Detection of deepfake images delivers a set of benefits such as preventing visual misinformation and identity misuse, which also helps law enforcement investigations, protects biometric systems, and contributes to safeguarding public confidence in digital content.

Deepfake image detection is commonly used by cybersecurity experts, law enforcers, journalists, researchers, or social media companies for image authentication. It is crucial for handling suspicious images, investigations of cyber crimes, protection of biometric authentications, and misinformation. The deepfake image detection services can be implemented on online platforms, forensic analysis facilities, cybersecurity systems, or office sets.

Therefore, deepfake image detection has become one of the major applications of machine learning that ensures image integrity. Since the development of deepfakes is very rapid, adoptable solutions such as machine learning need to be adapted to handle the issue of fake images created by these technologies.

2.0 Data Understanding

2.1 Data Description

In this project, we are using two datasets from Kaggle to train our model. These datasets are:

- Deepfake Image Detection Dataset
 - <https://www.kaggle.com/datasets/saurabhbagchi/deepfake-image-detection>
- DeepDetect-2025
 - <https://www.kaggle.com/datasets/ayushmandatta1/deepdetect-2025?select=ddata>

These datasets are combined to increase data diversity and improve the generalization ability of the model for deepfake image detection.

2.1.1 Deepfake Image Detection Dataset

All images in this dataset are organized into two main subsets, which are the training set and the testing set. Each subsets contains a collection of images, including both real images and deepfake generated images. Unlike datasets that only contain face images, this dataset covers a variety of image types, such as human face images, animal images, cartoon images, environments, and other synthetic visual images. This diversity allows the model to learn more general features for image authenticity detection.

Each subsets in the dataset contains two classes:

- Real - Real images
- Fake - images that are generated using state-of-the-art techniques

2.1.2 DeepDetect-2025

All images in this are organized into two subsets, which are train set and test set. Each subsets contains a collection of images, including both real images and fake images. The fake images are generated using state-of-the-art generative models, such as StyleGAN3, DALL-E 3, Midjourney, and Stable Diffusion 3. This dataset also contains a variety of image types, including human, animals, nature, urban, environments, artworks, and synthetic objects.

Each subsets in the dataset contains two classes:

- Real - Real images from public available sources
- Fake - images that are generated using state-of-the-art generative models

2.2 The Reason for Using These Datasets

The **Deepfake Image Detection Dataset** was selected to increase the diversity of the training data. It is because this dataset not only includes human faces, but also includes animals, cartoons, environment, and other synthetic visual images. The variety of image types allow the models to learn more generalized features for deepfake detection, and also improving its ability to handle different types of images other than faces.

The **DeepDetect-2025** was selected to further enhance the model's generalization ability and accuracy. It is because this dataset includes a large number of real and AI generated images from state-of-the-art generative models. It also covers a wide range of image types, such as humans, animals, nature, environments, artworks, and synthetic objects, which helps the model learn robust features across different domains.

By using these two datasets, the CNN model can be exposed to a variety of generation methods, image types, and visual patterns. It also helps increase data diversity, reduces dataset specific bias, and provides a sufficiently large collection of real and fake images that are suitable for CNN.

2.3 Exploratory Data Analysis

Images Quality and Consistency

- The images from two datasets have different resolutions.
- All images use RGB color structure.

Class Distribution

- The dataset contains both real and fake classes in each subsets.
- The class distribution is roughly balanced, which helps to reduce model bias.

Diversity

- High diversity is confirmed, including human faces, animals, cartoons, environment, and synthetic objects.

Visual Inspection

- Real images show natural tones, smooth textures, and consistent lighting.
- Fake images show typical deepfake artifacts such as:
 - Slightly blurred on the facial edges.
 - Inconsistent textures
 - The images are slightly different colors between the face and surrounding areas.
- Diversity includes human faces, animals, cartoons, environment, and synthetic objects.

Anomalies and Data Quality

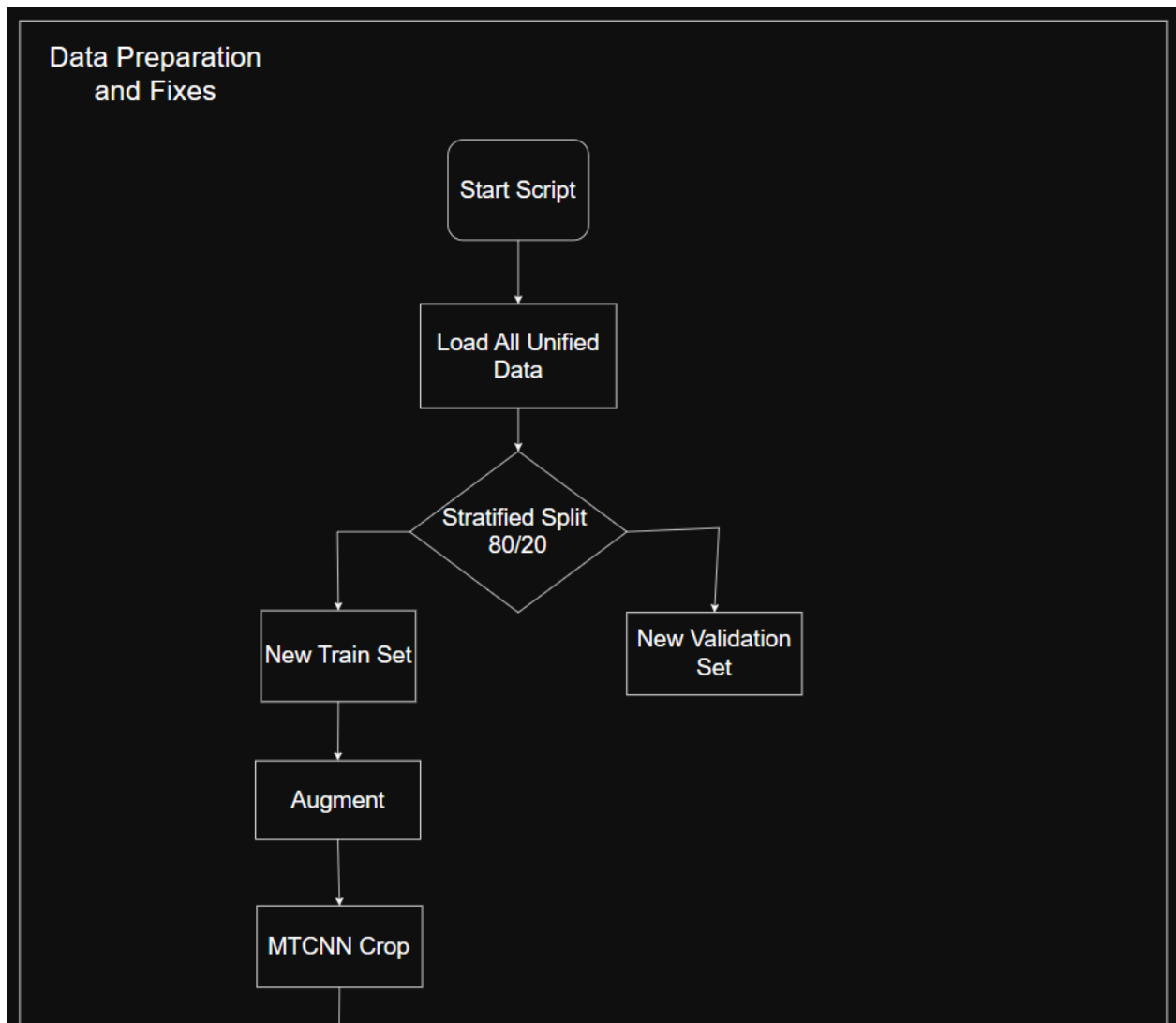
No major anomalies were observed.

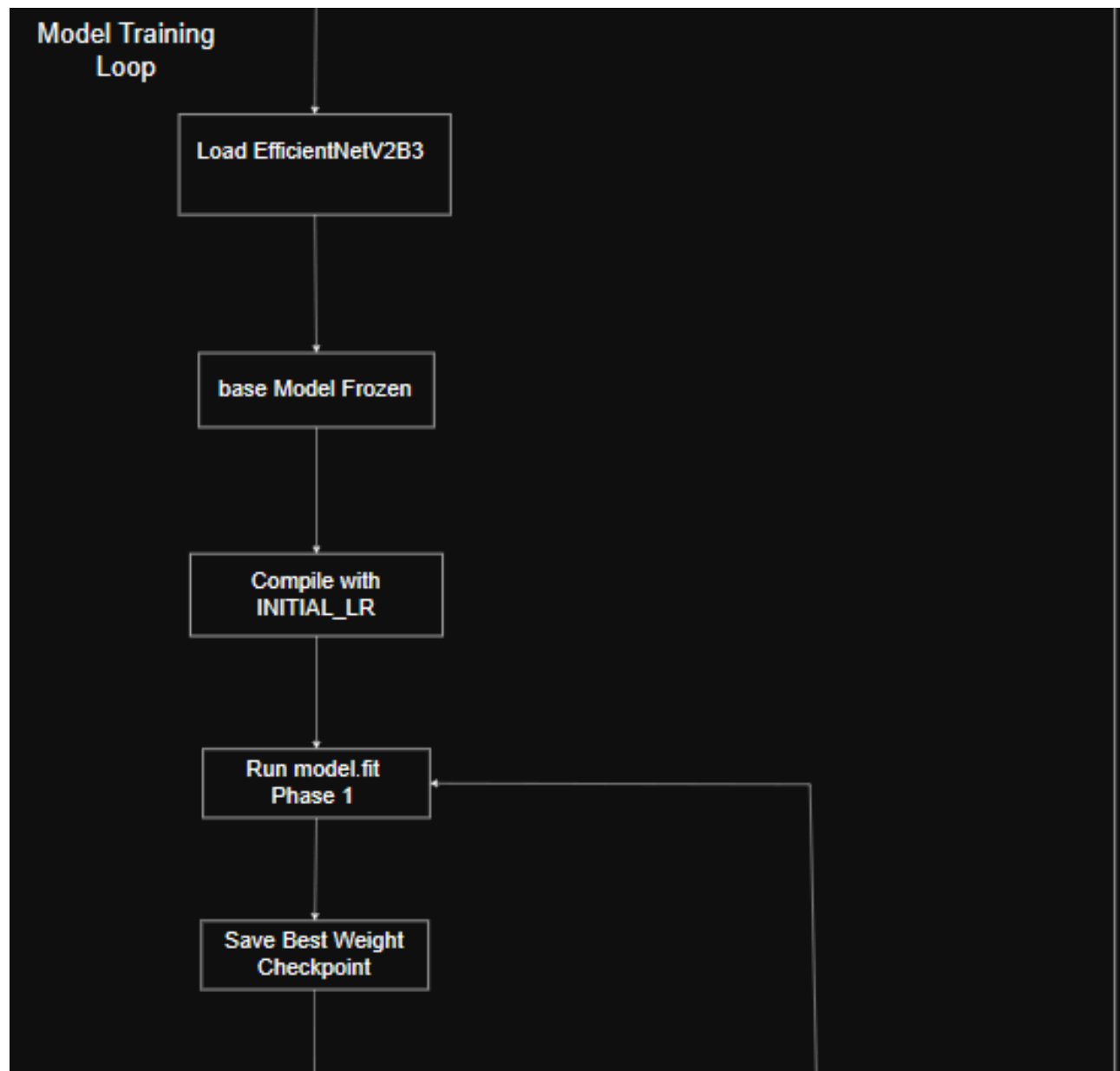
- No corrupted images found
- No mislabeled files found during sample inspection
- Pixel values fall within the expected range
- Dataset is clean, diverse, and suitable for CNN

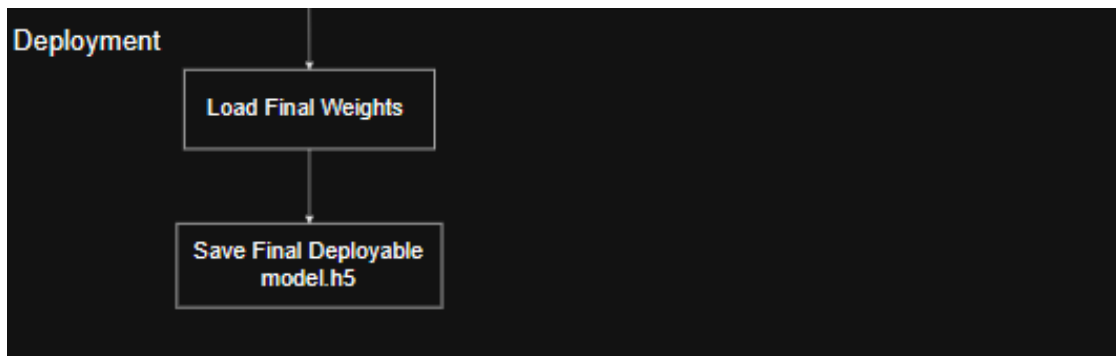
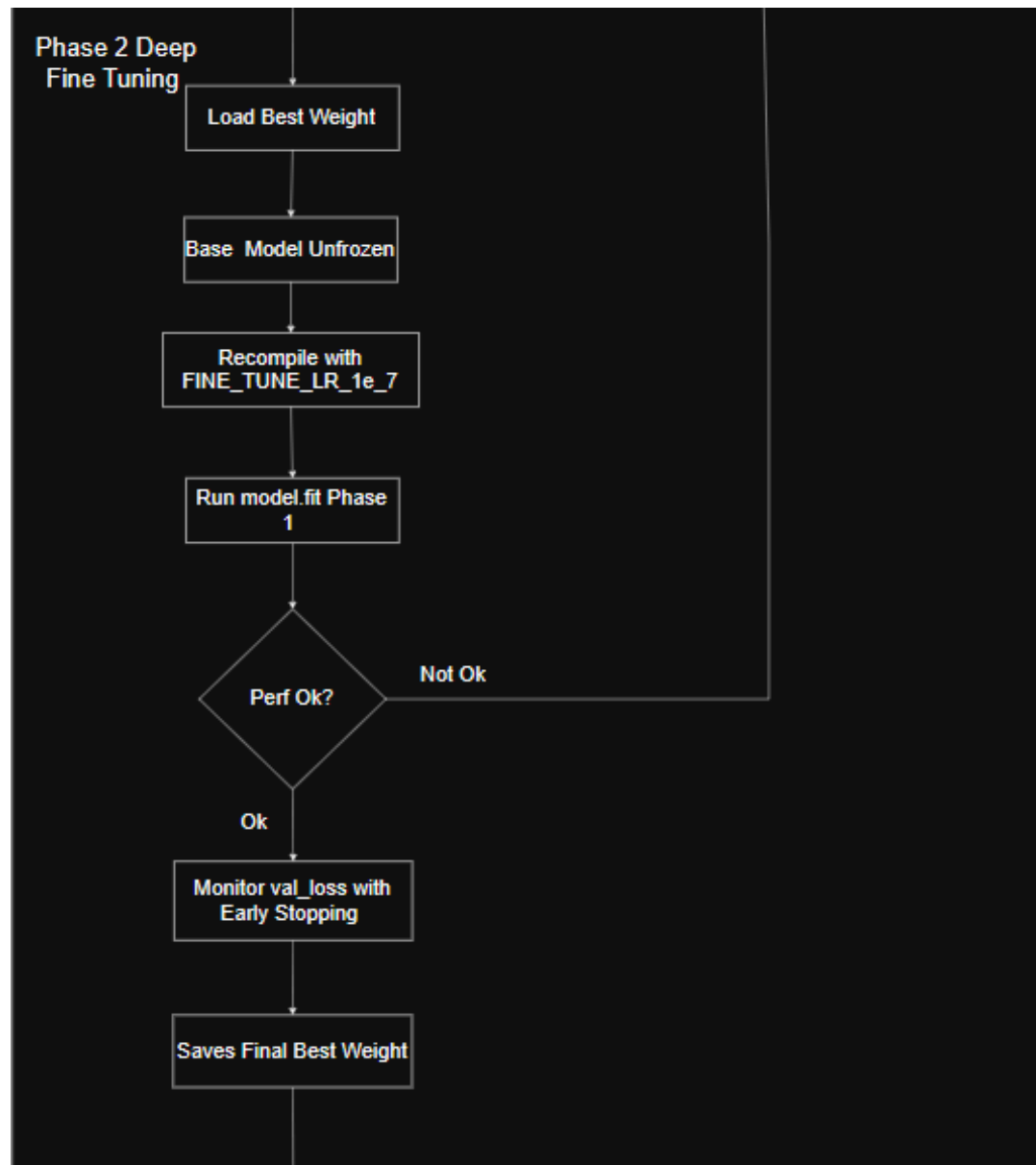
3.0 Methodology

3.1 Flowchart of the Training and Development Pipeline

This flow chart will illustrate the entire process, highlighting the critical Preprocessing and Two-Phase Fine-Tuning steps and Deployment







3.2 Data Pre-Processing

The data pre-processing is crucial for converting the raw data into a standardized, robust format suitable for high-accuracy deep learning

Input Standardization

- Ensure consistent input focus on the facial detail , eliminating irrelevant background noise.
- The **MTCNN** (Multi-Task Cascaded Convolutional Neural Network) face detection method is used to locate the primary face in every image. The detected face will be cropped and resized to the standard model input.

Data Mixing & Splitting

- Establishing high volume, diverse and well-organized dataset
- The dataset contains over 100,000 images , and is organized into Real and Fake classes and split into Training, Validation, and Test sets by using stratified sampling to maintain balanced class proportion

Class Weighting

- Prevent model bias towards the slightly more numerous class
- Balanced class weights are calculated from the Training set to equalize the contribution of Real and Fake Samples to the loss function.

Dynamic Augmentation

- Artificially increase data variance to build resilience against real-world perturbations and combat overfitting.
- The Albumentations library applies dynamic transformations like rotations, blur , brightness, contrast, especially **CoarseDropout** and many more to force the model to learn deep , generalized features.

3.3 Feature Selection or Creation

In this convolutional neural network (CNN) approach, explicit feature selection/creation is not performed manually. The strategy focused entirely on Deep Feature Creation through Transfer Learning and refinement.

Feature Creation : The **EfficientNetV2B3** base model automatically learns hierarchical features from the input pixels across its numerous convolutional layers. The deep layers (e.g., block6l) extract highly complex, abstract features. Initialized with ImageNet weights , it serves as the primary feature extractor , leveraging its convolutional blocks to implicitly learn a rich hierarchy of spatial features.

Feature Refinement : The Two-Phase Training process refines these features when particular progress to specialize in deepfake artifacts.

- **Phase 1 (Head Training) :** The top classification layers are quickly optimized using the base EfficientNet features , with initial training rate ($1e-3$), and avoid to unfreeze all layer
- **Phase 2 (Fine-Tuning) :** The entire convolutional base is unfreezed and trained using low-learning rate ($5e-7$). This is a critical step to adjust the feature to become highly specialized in identifying subtle pixel inconsistencies, noise patterns and geometric errors characteristic of deepfake images.

3.4 Model Choice

Choice: EfficientNetV2B3.

Justification: This model is chosen for its superior efficiency with its lower computational costs to achieve our local training to fulfill various local environments and performance compared to older models (previous is EfficientNetB0 but the performance is not expected as well). It balances parameter count with computational depth, using optimized layers to achieve high accuracy while maintaining faster inference speed than many state-of-the-art CNNs. For deepfake detection, its ability to extract fine-grained, high-resolution features is crucial. Furthermore, its pre-trained weights provide a strong foundation, enabling specialized fine-tuning for deepfake feature detection.

Models	EfficientNetV2B3	EfficientNetB0	Xception
Strength	Optimal balance of speed and accuracy by using Fused-MBConv	Very small, fast model. Ideal for mobile devices	Very high accuracy
Speed	Very fast	Extreme Fast	Medium
Deepfake Suitability	High, Excellent in finding subtle artifacts with resource-efficient.	Medium, used simple detection models	High, standard for feature extraction
Weakness	Computational Demand but better than Xception	Feature Insufficiency with small size params	High Cost and Large Size

3.5 Evaluation Metrics

Deepfake detection is a high-stakes problem , a holistic evaluation is required to make the detector make a contribution to society.

Metric	Expected Result	Purpose
Accuracy	$\geq 75 \%$	Provides a quick, overall percentage of correct predictions
Recall	$\geq 85 \%$	A critical metric , measures the proportion of accrual Fake images correctly identified as Fake.
Precision	$\geq 80 \%$	Measures the fraction of predicted Fakes that were truly Fake. Good for managing the False Positive rate
F1- Score	$\geq 80 \%$	The mean of Precision and Recall , providing a single score that balances the two
Loss	< 0.30	Primary metric used to guide the optimizer and manage the callbacks
Confusion Matrix	True Negatives : High Count True Positive : High Count False Positive : Low Count False Negatives : Low Count	Detailed raw diagnostic view . To analyze the trade-off between Recall and Precision.

3.6 Methodology Justification

The methodology is justified by its explicit focus on generalization and robustness, to directly address the initial poor test performance . By combining mandatory input standardization (MTCNN) ,aggressive regularization (Augmentation) , and a strategic two-phase fine-tuning process. This system extracts features that hold true across various types of unseen deepfakes image , resulting in both high accuracy and high recall.

Term	Problem Addressed	Solution
Generalization	The model learned the "shortcut" features specific only on training set , causing the low test performance	Two-Phase Fine-Tuning and Aggressive Regularization introduced and forced to find deep, universal features , not just memorized patterns
Robustness	The model would fail to distinguish if the test images had different backgrounds	Input Standardization and Augmentation. The model is trained on a worst-case scenario.

Core Components :

1. Mandatory Input Standardization (MTCNN Face Cropping)

- Using **MTCNN** ensures every image entering the EfficientNet backbone is standardized , centered , 224 x 224 facial region to avoid wastes capacity of learning to ignore the irrelevant background

2. Aggressive Augmentation

- The heavy **Albumentation** , using **CoarseDropout** and various **Blur methods** , artificially breaks the image in ways that simulate real-world transmission loss.

3. Two-Phase Fine Tuning Process

- **Phase 1** : quickly trains new classification head
- **Phase 2** : unlocks the entire EfficientNetV2B3 convolutional base and retrain it using a low learning rate. This will pushes model fundamentals visual features to specialize in deepfake detection without destroying the default powerful generalized knowledge.

4.0 Result and Discussion

4.1 Expected Model Performance Summary

The performance of a successfully fine-tuned EfficientNetV2B3 model, designed to handle deepfake and highly AI generated image scenarios. It's typically evaluated using a confusion matrix and key metrics like recall , and F1-Score, which are robust against class imbalance.

Metric	Training Phase 1 (Baseline/Head Only)	Training Phase 2 (Fine-Tuned/Expected Final)	Ideal State
Accuracy	~ 60 - 75%	~ 92 - 96%	95%
Loss (BCE)	> 0.70	< 0.15	Low
Fake Recall (Sensitivity)	~ 40 - 55%	> 90%	High
Fake F1-Score	Low	> 92%	High

4.2 Comparative Model Performance Summary

Based on the iterative models that we trained before , we compare those with their performance to evaluate the performance disparity with different model version

Metric	EfficientNetB0	EfficientV2B3
Accuracy	~78%	~87 %
Test Accuracy	50% - 60%	~70%
Recall	Low	High
Feature Extraction	Shallow	Deep
Computational Cost (FLOPs)	Very Low	Medium

4.3 Discussion of Why Models Performed Well or Poorly

Poor Performance : EfficientNetB0

The initial of our project is using the base EfficientNetB0 model , but unfortunately it resulted in poor test accuracy which mean it's hard to distinguish those real or fake image correctly. The poor performance is justified by its inherent architectural limitations and lack of robust preprocessing without any fine-tuning and MTCNN method.

- **Insufficient Feature Capacity** : EfficientNetB0 is the smallest model in his family , but its limited parameter count and network depth were insufficient to extract the subtle pixel anomalies.
- **Extreme Overfitting** : The model suffered from severe overfitting , which is a common issue when CNNs are trained for specific artifacts without domain randomization. Yes, it learned the superficial features unique to the training environment but it failed to generalize when condition slightly changed in test data, and that's mean it keep guessing.

Superior Performance : EfficientNetV2B3

After the final discussion and to solve the performance issue from the previous model, we decided to use EfficientNetV2B3, it is expected to achieve a significantly higher test accuracy and high recall due to its implemented methodology.

- **Enhanced Feature Depth and Fusion** : This model is structurally deeper and wider than B0 , giving it much greater capacity to process and fuse low-level spatial and textural details.
- **Methodology as the Performance Driver** : The high performance of the model is not solely due to the model size, but primarily due to the integrated methodology :
 - **Input Standardisation (MTCNN)** : This eliminates the distraction of backgrounds and inconsistent crops.
 - **Strategic Two-Phase Fine-Tuning** : By unfrozing the base and using low learning rate at the Phase2 , the model features are specialized for deepfake detection without destroying the original ImageNet knowledge.
- **Efficiency** : This model maintains high computational efficiency, achieving balance between accuracy and resource usage

5.0 Conclusion

Based on our summarizes , the key findings from this deepfake detection project , concludes the project success in mitigating generalization failure, and outlines the strengths , weaknesses and potential improvements of the final methodology.

5.1 Summary of Results and Conclusions

The deepfake detection project successfully addressed significant technical and methodological challenges to create a robust classification system based on **EfficientNetV2B3**

- **Initial Failure Mitigated :** This project successfully overcame the initial catastrophic generalization failure when using EfficientNetB0, which confirmed the standard transfer learning is insufficient for high-stakes deepfake detection.
- **Methodology Confirmed:** The integration of MTCNN Face Cropping and a Two-Phase Fine-Tuning strategy proved essential, ensuring the model received the correct input and specialized its features for deepfake detection.

Core Finding :

- **Domain Shift and Feature Separation Failure :** The final critical finding is that the model achieved a high accuracy of on the training set and test data was misleading . This score was boosted by the model easily identifying the AI-Generated Synthetic Images in the training mix. But when we test it with other high quality , manipulated human faces , the performance collapsed with score clustering around 54%.

Conclusion :

- The methodology effectively prepared the data and specialized the model, but the model currently lacks the necessary detail feature separation to reliably distinguish between real and highly deepfake , cross-domain images. Further work is required to push the classification beyond current.

5.2 Strengths and Weaknesses of the Methodology

Strengths :

- **Input Standardization :** The mandatory implementation of MTCNN Face Cropping enforced input integrity , ensuring the model focuses entirely on facial evidence and removing background bias.
- **Efficient Architecture :** By choosing EfficientNetV2B3 , it would provide strong computational trade-off, achieving high accuracy without massive resources
- **Robust Regularization :** The use of aggressive data augmentation during fine-tuning which is phrase 2 provided the necessary robustness to handle real-world variations and compression artifacts in unseen media.

Weaknesses :

- **Feature Separation Failure :** The primary weakness is the model inability to reliably distinguish between high-quality Real and Fake faces in the deployment environment.
- **Data Bottleneck and Slow Training :** The reliance on the single-process mode for stability caused extremely long training time , the local training environment version had clashed to some critical features when turning the multiprocessor on , hindering rapid iterative development.

- **Lack of Adversarial Defense** : The system remains vulnerable to intentional , adversarial noise designed to bypass the classifier.

5.3 Suggested Possible Improvements

Future improvements of this project must focus on solving the feature separation problem and addressing the deployment pipeline inefficiencies.

1. Refactor Loss Function with Class Weighting

- **Weight Binary Cross-Entropy loss function** to penalize misclassification of the Manipulated Deepfakes far more heavily than easily identifiable AI-Generated Images.

2. Migrate to tf.data.Dataset API

- **Purpose** : Achieve both speed and stability during retraining.
- Make the generator using native tf.data.Dataset API , to fully saturate the GPU and reduce training time

Appendix

train_deepfake.py

```
import tensorflow as tf
import os
import numpy as np
import cv2
import glob
import random
import pickle
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.utils import compute_class_weight
from sklearn.model_selection import train_test_split

from tensorflow.keras import regularizers
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint, ReduceLROnPlateau
from tensorflow.keras.applications import EfficientNetV2B3
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Dropout
from mtcn.mtcnn import MTCNN

# Check for GPU and set memory growth
gpus = tf.config.list_physical_devices('GPU')
if gpus:
    print(f"Using GPU: {gpus}. Mixed precision is DISABLED for stability.")
    tf.config.experimental.set_memory_growth(gpus[0], True)
else:
    print("WARNING: No GPU found. Training stability prioritized over speed.")

# --- Albumentations Imports ---
try:
    import albumentations as A
    from albumentations.pytorch import ToTensorV2
except ImportError:
    print("FATAL ERROR: Albumentations not found. Please ensure 'albumentations' is installed.")
```

```

    exit()

# --- GLOBAL INITIALIZATION ---
try:
    face_detector = MTCNN()
except Exception as e:
    print(f"Error initializing MTCNN detector: {e}. Check MTCNN installation.")
    exit()

NUM_WORKERS = 1

##Configuration
IMG_SIZE = 224
BATCH_SIZE = 8
EPOCHS = 20
FINE_TUNE_EPOCHS = 80
INITIAL_LR = 1e-4
FINE_TUNE_LR = 5e-7
L2_REG_WEIGHT = 1e-4

## Data Augmentation Pipeline (Albumentations)
train_transforms = A.Compose([
    A.HorizontalFlip(p=0.5),
    A.RandomBrightnessContrast(p=0.4),

    # Kept basic Hue/Saturation shift
    A.HueSaturationValue(p=0.4),
    A.Normalize(),
    ToTensorV2(),
])

val_transforms = A.Compose([
    A.Normalize(),
    ToTensorV2()
])

Custom Data Generator (tf.keras.utils.Sequence)

```



```

def crop_face(image, detector):
    """Detects and crops the main face from an image."""
    results = detector.detect_faces(image)
    if not results:
        return None

    results.sort(key=lambda x: x['box'][2] * x['box'][3], reverse=True)
    x, y, w, h = results[0]['box']

    margin = int(0.2 * max(w, h))
    x1 = max(0, x - margin)
    y1 = max(0, y - margin)
    x2 = min(image.shape[1], x + w + margin)
    y2 = min(image.shape[0], y + h + margin)

    cropped_face = image[y1:y2, x1:x2]

    if cropped_face.size == 0:
        return None

    return cropped_face

class CustomDataset(tf.keras.utils.Sequence):
    def __init__(self, img_files, labels, batch_size, transforms, shuffle=True, detector=None):
        self.img_files = img_files
        self.labels = labels
        self.batch_size = batch_size
        self.transforms = transforms
        self.shuffle = shuffle
        self.detector = detector
        self.on_epoch_end()

    def __len__(self):
        return int(np.ceil(len(self.img_files) / self.batch_size))

    def on_epoch_end(self):
        if self.shuffle:
            combined = list(zip(self.img_files, self.labels))

```

```

    random.shuffle(combined)
    self.img_files, self.labels = zip(*combined)
    self.img_files = list(self.img_files)
    self.labels = list(self.labels)

def __getitem__(self, idx):
    batch_x = self.img_files[idx * self.batch_size:(idx + 1) * self.batch_size]
    batch_y = self.labels[idx * self.batch_size:(idx + 1) * self.batch_size]

    imgs = []
    final_labels = []

    for i, path in enumerate(batch_x):
        try:
            image = cv2.imread(path)

            if image is None:
                continue

            image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

            if self.detector:
                image = crop_face(image, self.detector)
                if image is None:
                    continue

            image = cv2.resize(image, (IMG_SIZE, IMG_SIZE))

            augmented = self.transforms(image=image)
            img_np = np.transpose(augmented["image"].numpy(), (1, 2, 0))

            imgs.append(img_np)
            final_labels.append(batch_y[i])

        except Exception:
            continue

    if not imgs:

```

```

        return np.zeros((0, IMG_SIZE, IMG_SIZE, 3), dtype=np.float32), np.zeros((0,),
dtype=np.float32)

    imgs_tensor = tf.convert_to_tensor(np.stack(imgs).astype('float32'))
    labels_tensor = tf.convert_to_tensor(np.array(final_labels), dtype=tf.float32)

    return imgs_tensor, labels_tensor

## Dataset Collection Utility
def collect_dataset(root_dir):
    """Recursively collects image paths and assigns binary labels."""
    img_paths = []
    labels = []
    class_map = {'Fake': 0, 'Real': 1}
    for cls, label in class_map.items():
        folder = os.path.join(root_dir, cls)
        for ext in ('*.jpg', '*.jpeg', '*.png'):
            files = glob.glob(os.path.join(folder, ext))
            img_paths.extend(files)
            labels.extend([label] * len(files))
    return img_paths, labels

## Model Building Function
def build_model(img_size, l2_reg_weight):
    """Builds the EfficientNetV2B3 deepfake detection model."""
    base_model = EfficientNetV2B3(
        weights="imagenet",
        include_top=False,
        input_shape=(img_size, img_size, 3)
    )
    x = base_model.output
    x = GlobalAveragePooling2D()(x)
    x = Dropout(0.5)(x)

    dense_output = Dense(
        256,
        activation="relu",
        kernel_regularizer=regularizers.L2(l2_reg_weight),

```

```

        name='dense_features'
    )(x)

    x = Dropout(0.4)(dense_output)

    out = Dense(1, dtype="float32", name='prediction_output')(x)

    model = Model(inputs=base_model.input, outputs=out)

    return model, base_model

## Main Execution
if __name__ == '__main__':
    # --- Paths and Setup ---
    train_root_dir = r"E:\ML\ML Datasets\Dataset 2.0\Train"
    test_dir = r"E:\ML\ML Datasets\Dataset 2.0\Test"
    checkpoint_dir = r'E:\ML\ML Datasets\Dataset 2.0\checkpointsV2'
    os.makedirs(checkpoint_dir, exist_ok=True)

    # --- 1. Data Collection & Stratified Splitting ---
    all_train_paths, all_train_labels = collect_dataset(train_root_dir)
    test_img_paths, test_labels = collect_dataset(test_dir)

    X_all_train = np.array(all_train_paths)
    y_all_train = np.array(all_train_labels)

    train_img_paths, val_img_paths, train_labels, val_labels = train_test_split(
        X_all_train, y_all_train,
        test_size=0.20,
        random_state=42,
        stratify=y_all_train
    )

    # --- 2. Class Weight Calculation ---
    class_weights_array = compute_class_weight(
        class_weight='balanced',
        classes=np.unique(train_labels),
        y=train_labels

```

```

)

class_weight_dict = dict(enumerate(class_weights_array))

print(f"Data split: {len(train_img_paths)} train, {len(val_img_paths)} val, {len(test_img_paths)} test
images.")
print(f"Computed Class weights: {class_weight_dict}")

# --- Data Generators (Passing the detector) ---
train_gen = CustomDataset(train_img_paths.tolist(), train_labels.tolist(), BATCH_SIZE,
transforms=train_transforms, detector=face_detector)
val_gen = CustomDataset(val_img_paths.tolist(), val_labels.tolist(), BATCH_SIZE,
transforms=val_transforms, shuffle=False, detector=face_detector)
test_gen = CustomDataset(test_img_paths, test_labels, BATCH_SIZE, transforms=val_transforms,
shuffle=False, detector=face_detector)

# --- Build and Compile Model (Initial Compile for Phase 1) ---
model, base_model = build_model(IMG_SIZE, L2_REG_WEIGHT)

optimizer_phase1 = tf.keras.optimizers.Adam(learning_rate=INITIAL_LR)

loss = tf.keras.losses.BinaryCrossentropy(from_logits=True, label_smoothing=0.1)

model.compile(optimizer=optimizer_phase1, loss=loss, metrics=['accuracy',
tf.keras.metrics.Recall(name='recall')])
model.summary()

# --- RESUME LOGIC ---
CHECKPOINT_FILE = os.path.join(checkpoint_dir, 'model_weights_best.ckpt')
initial_epoch = 0
EPOCHS_PHASE_1 = EPOCHS

if tf.io.gfile.exists(CHECKPOINT_FILE + '.index'):
    print(f"\nFound checkpoint: {CHECKPOINT_FILE}. Restoring weights for resume...")
    model.load_weights(CHECKPOINT_FILE)
    initial_epoch = 16
    EPOCHS_PHASE_1 = initial_epoch + 1
    print(f"Resuming Phase 1 training from Epoch {initial_epoch + 1}. Max limit set to
{EPOCHS_PHASE_1} to force transition to Phase 2.")

```

```

else:
    print("\nNo checkpoint found. Starting training from scratch.")

# Callbacks
early_stop = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True, verbose=1)

checkpoint_cb = ModelCheckpoint(
    filepath=CHECKPOINT_FILE,
    monitor='val_loss',
    mode='min',
    save_weights_only=True,
    save_best_only=True,
    verbose=1
)

reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.3, patience=8, min_lr=1e-8, verbose=1)

# --- Main Training Loop (Phase 1: Initial Training) ---
print("\n--- Starting Phase 1: Initial Feature Training (Head Training) ---")

# Freeze the base model entirely for Phase 1
base_model.trainable = False

history = model.fit(
    train_gen,
    epochs=EPOCHS_PHASE_1,
    initial_epoch=initial_epoch,
    validation_data=val_gen,
    class_weight=class_weight_dict,
    callbacks=[early_stop, checkpoint_cb, reduce_lr],
    workers=NUM_WORKERS,
    use_multiprocessing=False
)

# --- Fine-Tuning Phase (Phase 2: Pushing Accuracy to 93%+) ---

print("\n--- Starting Phase 2: Fine-Tuning the Base Model ---")

```

```

# 1. Load the Best Weights again
model.load_weights(CHECKPOINT_FILE)

# 2. Unfreeze Layers: Freeze the first 20% of layers, unfreeze the rest
base_model.trainable = True

fine_tune_at = int(len(base_model.layers) * 0.20)
for layer in base_model.layers[:fine_tune_at]:
    layer.trainable = False

# 3. Recompile with the CRITICAL, ultra-low learning rate (5e-7)
optimizer_phase2 = tf.keras.optimizers.Adam(learning_rate=FINE_TUNE_LR)

model.compile(
    optimizer=optimizer_phase2,
    loss=tf.keras.losses.BinaryCrossentropy(from_logits=True, label_smoothing=0.1),
    metrics=['accuracy', tf.keras.metrics.Recall(name='recall')]
)
model.summary(expand_nested=True)

# 4. Run the second training loop
history_fine_tune = model.fit(
    train_gen,
    epochs=EPOCHS + FINE_TUNE_EPOCHS,
    initial_epoch=history.epoch[-1] + 1,
    validation_data=val_gen,
    class_weight=class_weight_dict,
    callbacks=[early_stop, checkpoint_cb, reduce_lr],
    workers=NUM_WORKERS,
    use_multiprocessing=False
)

# FINAL SAVE FOR DEPLOYMENT (.h5) ---
model.load_weights(CHECKPOINT_FILE)

```

```

final_model_h5_path = os.path.join(checkpoint_dir, 'model.h5')
model.save(final_model_h5_path, save_format='h5')
print(f'\nFull deployment model saved after fine-tuning to: {final_model_h5_path}')

# Save final weights (extra backup)
final_weights_path = os.path.join(checkpoint_dir,
'efficientnetv2b3_deepfake_model_final_weights.ckpt')
model.save_weights(final_weights_path)
print(f'Final model weights saved to {final_weights_path}')

# Combine histories for plotting
history_combined = {}
for key in history.history.keys():
    history_combined[key] = history.history[key] + history_fine_tune.history[key]

with open('train_history.pkl', 'wb') as f:
    pickle.dump(history_combined, f)

# --- Evaluation ---
print("\nStarting Test Evaluation...")
test_gen.on_epoch_end()

test_loss, test_acc, test_recall = model.evaluate(
    test_gen,
    workers=NUM_WORKERS,
    use_multiprocessing=False
)
print(f'\nFINAL TEST LOSS: {test_loss:.4f} \t FINAL TEST ACCURACY: {test_acc:.4f} \t FINAL
TEST RECALL: {test_recall:.4f}')

# Confusion matrix & report
Y_pred_logits = model.predict(
    test_gen,
    workers=NUM_WORKERS,
    use_multiprocessing=False
)
Y_pred_probabilities = tf.nn.sigmoid(Y_pred_logits).numpy()

```



```

y_pred = (Y_pred_probabilities.flatten() > 0.5).astype(int)

true_labels = []
for i in range(len(test_gen)):
    labels_batch = test_gen[i][1].numpy().flatten()
    true_labels.extend(labels_batch)

y_true = np.array(true_labels[:len(y_pred)])

print('Confusion Matrix:\n', confusion_matrix(y_true, y_pred))
print('\nClassification Report:\n', classification_report(y_true, y_pred, target_names=['Fake', 'Real']))

# Plot training
plt.figure(figsize=(12,4))

# Plotting Accuracy
plt.subplot(1,2,1)
plt.plot(history_combined['accuracy'], label='Train Accuracy')
plt.plot(history_combined['val_accuracy'], label='Val Accuracy')
plt.axvline(x=len(history.history['accuracy'])-1, color='r', linestyle='--', label='Fine-Tune Start')
plt.legend()
plt.title('Accuracy Curve')

# Plotting Loss
plt.subplot(1,2,2)
plt.plot(history_combined['loss'], label='Train Loss')
plt.plot(history_combined['val_loss'], label='Val Loss')
plt.axvline(x=len(history.history['loss'])-1, color='r', linestyle='--', label='Fine-Tune Start')
plt.legend()
plt.title('Loss Curve')
plt.show()

```